

Informe de Backend 24/03/2026

-Proyecto ATALANTA

Informe de Arquitectura Cloud y Análisis Estructural Backend

Proyecto: Atalanta Gestor de Incidencias

Rol: Arquitecto de Sistemas Cloud / Especialista en Ciberseguridad Backend

Fecha: 24 de Marzo de 2026

1. Composición de la Infraestructura

Naturaleza del Backend

El proyecto Atalanta opera bajo una **Arquitectura Híbrida Transicional**, fuertemente apalancada en un modelo **Backend-as-a-Service (BaaS)** mediante Supabase.

Aunque existe un servidor **Node.js/Express** independiente en el directorio /gestor_incidencias configurado con helmet, cors, express.json(), jsonwebtoken y mysql2 (lo que indica un diseño Legacy o una API privada en desarrollo), la **lógica principal de negocio y producción recae en Supabase**. El frontend consume directamente la base de datos de Supabase vía SDK (modelo Client-to-BaaS), puenteando en gran medida el servidor Node.js.

Stack Tecnológico Principal

- **Base de Datos y API REST:** PostgreSQL orquestado por **Supabase** (PostgREST para exponer DDL/DML como REST API).
 - **Autenticación y Sesiones:** Supabase Auth (Basado en el motor GoTrue de Netlify). Maneja JWT subyacente.
 - **Backend Monolítico (Secundario/Legacy):** Node.js 20+ con Express 5.2.1, encriptación manual con Bcrypt 6.0 y tokens JWT 9.0.
 - **Seguridad Frontera Node:** Helmet (Cabeceras HTTP seguras), CORS, xss.
 - **Frontend:** Vanilla JavaScript/HTML5 servido estáticamente (potencialmente por Express app.use static).
-

2. Mapa de Funciones Principales

Gestión de Identidades (Identity Management)

1. **Registro:** El cliente solicita la creación a Supabase Auth signUp(). La contraseña se cifra inmediatamente en el servidor de identidad. La confirmación desencadena (vía inserción manual del cliente en el front) la persistencia del UUID en la tabla pública profiles con rol predeterminado.
2. **Login y Sesión:** El método signInWithPassword() devuelve una sesión JWT. La persistencia es delegada

al localStorage o sessionStorage administrado automáticamente por el cliente oficial @supabase/supabase-js.

Lógica de Incidencias (Ticketing Engine)

El sistema mapea las entidades directamente a relaciones RESTful:

- **CREATE:** Clientes insertan filas en tickets uniendo su auth.uid() como cliente_id.
- **READ:** Trabajadores/Admins solicitan JOINS implícitos al motor de DB (ej. cliente:profiles!tickets...) para resolver el nombre del cliente y trabajador simultáneamente.
- **UPDATE:** Modificaciones de estado (Abierto, En progreso, Resuelto) permitidas para el perfil asignado (trabajador_id).
- **DELETE:** (Histórico inmutable) No existe lógica de exposición DML para borrar tickets, garantizando auditorías estables.
- **Comentarios (Sub-nodo):** Inserciones referenciadas al ticket_id limitadas cronológicamente para añadir bitácoras técnicas.

Sistema de Roles y Jerarquía

Existen 4 sub-escalas de privilegios: cliente, trabajador, jefe, admin.

- **Validación de acción:** El backend NO valida de acuerdo a la cadena de texto almacenada en el

LocalStorage. En cada petición, el JWT es decodificado en Postgres (función `auth.uid()`) y cruzado contra la tabla interna `profiles` u operando en validación DML de RLS.

- **Redirección de flujo (View-Routing):** El dashboard re-enruta la UI tras el login hacia `/dashboard/admin.html` o `/dashboard/cliente.html` facilitando la Experiencia de Usuario, pero asumiendo que un ingreso manual a la URL bloquea su vista sin RLS activo.

3. Análisis de Funciones de Base de Datos (RPC y Triggers)

Funciones PL/pgSQL: `asignar_rol(p_user_id, p_nuevo_rol)`

- **Flujo Condicional Externo:** Es el único portal DML expuesto explícitamente para alterar roles.
- **Input:** El UUID del destino y la cadena de texto del nuevo rol.
- **Ejecución Lógica:** Operador `SECURITY DEFINER`. Obliga al motor a consultar qué rol ostenta el ejecutante original extrayendo la fila de su `auth.uid()`.
- **Output:** Si la variable `v_invocador_rol == 'admin'`, ejecuta `UPDATE profiles`. Si no, expulsa `RAISE EXCEPTION`.

Patrón Trigger Ausente pero Recomendable

Actualmente, el registro empuja la autenticación y, seguidamente en la promesa del JS, un `.insert()` para poblar la tabla `profiles`.

- **Vulnerabilidad Potencial (Carrera / Interrupción):** Si el navegador se cierra tras el paso 1, el `Auth_user` se crea pero el `Profile` no.
- **Reingeniería recomendada:** Crear un Postgres Trigger `after_auth_insert` para que siempre que Gotrue cree una identidad, PL/pgSQL inserte una fila por defecto en `profiles` con rol cliente.

4. Seguridad y Endpoints

Cortafuegos RLS (Protección de Capas)

En la topología BaaS, la API está 100% abierta a Internet público. La seguridad es binaria: Token no válido vs Token válido. Si el token es válido, las tablas (`profiles`, `tickets`, `comentarios`) activan políticas de Row Level Security. El motor evalúa lógicamente cláusulas `USING (auth.uid() = XXX)` o `USING (EXISTS (SELECT 1 FROM profiles WHERE rol = 'admin'))` en milisegundos. Esta muralla es el eje central anti-penetración actual.

Gestión y Contención de Errores

Supabase devuelve objetos estándar `{ data, error }`. Los scripts del front han sido fortificados para atrapar fallos lógicos (`if (error)`) y canalizarlos a un `console.error` silencioso.

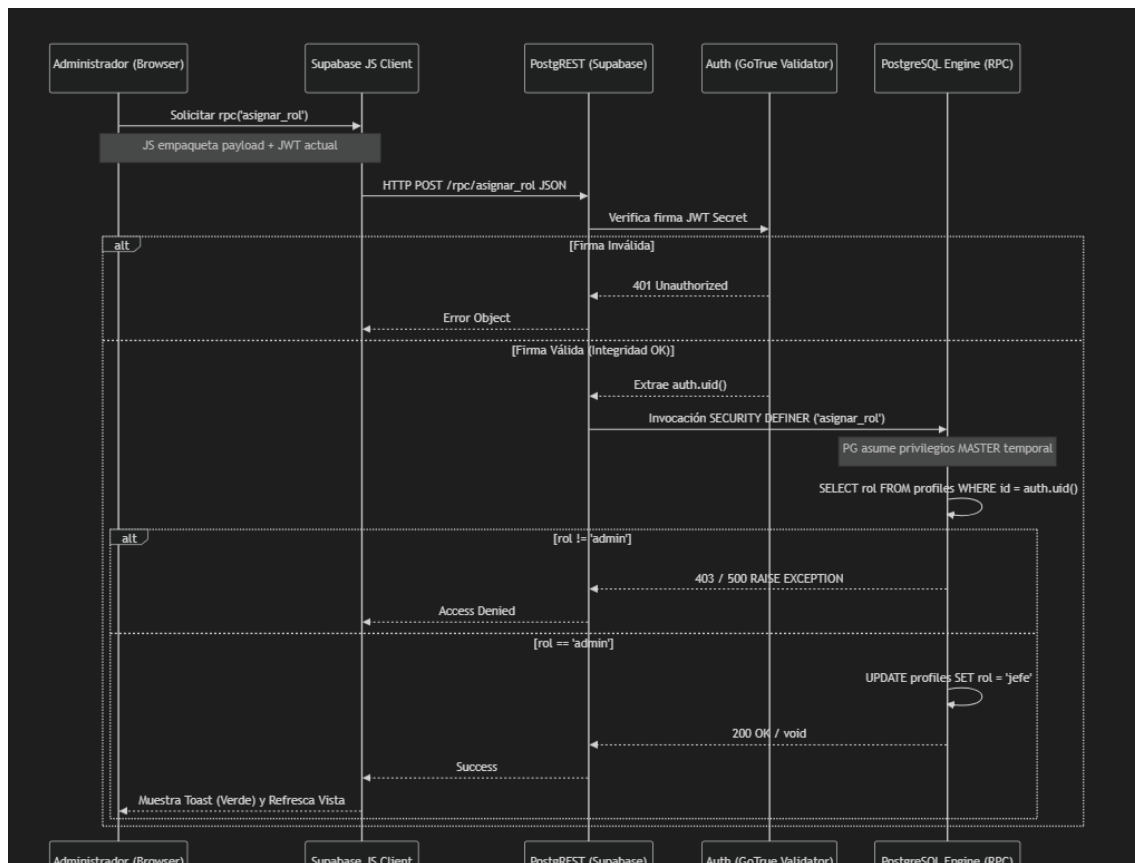
(sin emitir stacks de volcado de memoria a la consola nativa), acoplado a un "Toast" de notificación amigable al usuario (UX) para interrupciones no autorizadas, ocultando la topología relacional al atacante.

Integraciones y Comunicación Externa

Se observa la integración asíncrona explícita con EmailJS (plantillas referenciadas en `atalanta.js` e inicializadas con Public Key). El flujo delega el SMTP al proveedor de terceros. Debido a la arquitectura Frontend-Direct-BaaS, los correos corporativos de notificación no pasan por colas del backend Node, sino mediante invocaciones POST transparentes del navegador al endpoint genérico de EmailJS.

5. Diagrama de Flujo: Request Lifecycle (Ciclo de Vida de una Petición)

El siguiente ciclo detalla la operación de elevación de privilegios (*Cambiar Rol de Trabajador a Jefe*):



Cierre Arquitectónico

El backend actual posee una fuerte dualidad. Mientras el servidor backend monolítico de Node se encuentra en estado durmiente o para fines específicos, el cluster en Supabase orquesta masivamente las operaciones operando dentro de una cápsula estricta de BaaS. Esta decisión ahorra masivamente costos DevOps y de latencia a cambio de entregar total acoplamiento a las especificaciones relacionales (T-SQL, PL/pgSQL).